

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA
ESCUELA DE CIENCIAS Y SISTEMAS

INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 1 (IPC1)

INFORME TÉCNICO Y DOCUMENTACIÓN

ACTIVIDAD 5: MODELADO ORIENTADO A OBJETOS E INTERFAZ GRÁFICA CON
JAVA SWING

Estudiante:	Cristopher Sic
Curso:	Introducción a la Programación y Computación 1
Semestre:	Segundo Semestre 2026
Tecnologías:	Java SE 21, Java Swing Puro, Apache Maven
Fecha:	Septiembre 2026

1. Introducción al Proyecto

El presente informe documenta el desarrollo de la **Actividad 5** correspondiente al curso *Introducción a la Programación y Computación 1*. El objetivo principal de la actividad consiste en aplicar de forma integral los fundamentos de la Programación Orientada a Objetos (POO), construyendo un sistema de escritorio completo en Java mediante el uso exclusivo de la biblioteca gráfica **Java Swing por código puro**, sin recurrir a generadores automáticos de interfaz gráfica (drag-and-drop).

La aplicación integra dos núcleos operativos principales: en primer lugar, un módulo de gestión académica que permite la captura, vinculación y reporte estructurado de **Cursos universitarios** y **Tareas asignadas**; en segundo lugar, un módulo científico de **Conversión de Temperatura** entre grados Celsius, Fahrenheit y Kelvin, dotado de validaciones defensivas exhaustivas con control de excepciones y retroalimentación modal visual mediante JOptionPane.

2. Sección Teórica (Investigación)

2.1. Programación Orientada a Objetos: Encapsulamiento y Abstracción

La Programación Orientada a Objetos (POO) estructura los sistemas de software alrededor de entidades que encapsulan datos y comportamientos. La **abstracción** permite modelar las propiedades esenciales de un dominio excluyendo detalles no pertinentes. Por su parte, el **encapsulamiento** protege la integridad interna del estado del objeto ocultando sus campos mediante el modificador `private` y restringiendo su lectura y mutación a métodos públicos getters y setters. Esto evita corrupciones accidentales de datos y centraliza las reglas de validación en una única capa.

2.2. Biblioteca Gráfica Java Swing y Arquitectura de Contenedores

Java Swing es un conjunto de herramientas de interfaz gráfica ligera parte de las *Java Foundation Classes (JFC)*. Swing sigue una arquitectura jerárquica de contenedores: contenedores de nivel superior como JFrame albergan paneles intermedios JPanel y JScrollPane, que a su vez distribuyen componentes atómicos como JLabel, JTextField, JButton, JComboBox y JTextArea. El desarrollo por **código puro** garantiza independencia de entornos IDE específicos, código limpio sin metadatos residuales y un dominio absoluto sobre la disposición dimensional y el ciclo de vida de los componentes.

2.3. Modelo de Delegación de Eventos en Java (ActionListener)

El manejo de interacción en Swing implementa el patrón de diseño *Observer* mediante el modelo de delegación de eventos. Los componentes emisores (como un botón) disparan un `ActionEvent` al ser accionados. El programador registra escuchadores que implementan la interfaz funcional `ActionListener`, cuyo método `actionPerformed(ActionEvent e)` encapsula la lógica de respuesta. El empleo de clases anónimas permite vincular limpiamente la acción del usuario con la lógica de negocio sin dispersar el flujo de control.

2.4. Validación Defensiva de Datos y Manejo de Excepciones

La programación defensiva asume que las entradas del usuario pueden ser erróneas o maliciosas. El sistema valida la presencia de datos mediante `trim().isEmpty()` y verifica tipos de datos numéricos capturando la excepción en tiempo de ejecución `NumberFormatException` a través de bloques `try-catch`. Adicionalmente, se protege la lógica matemática frente a valores especiales como `Double.isNaN()` y `Double.isInfinite()` para evitar resultados anómalos.

2.5. Diálogos Modales y Retroalimentación con JOptionPane

La retroalimentación oportuna es indispensable en la Experiencia de Usuario (UX). La clase JOptionPane despliega diálogos modales que suspenden temporalmente el flujo de la aplicación hasta que el usuario confirma la notificación. Se emplean tres niveles semánticos: INFORMATION_MESSAGE para confirmaciones de éxito, WARNING_MESSAGE para advertencias preventivas (como casillas en blanco o precondiciones incumplidas) y ERROR_MESSAGE para fallas operativas o datos numéricos no procesables.

2.6. Colecciones Dinámicas en Memoria (ArrayList vs. Arreglos Estáticos)

A diferencia de los arreglos tradicionales de tamaño fijo (T[]), la clase ArrayList<T> ofrece un arreglo dinámico redimensionable que crece conforme se insertan elementos. En aplicaciones académicas donde el número de cursos y tareas no es constante, ArrayList optimiza el uso de memoria, elimina el riesgo de desbordamientos (ArrayIndexOutOfBoundsException) y provee métodos eficientes como add(), get() y size().

3. Diseño y Arquitectura de Clases

La solución se estructuró bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC), asegurando bajo acoplamiento y alta cohesión:

Capa	Clase	Responsabilidad Técnica
Modelo	Curso.java	Encapsula código, nombre y tutor con validaciones y métodos de formato.
Modelo	TareaAcademica.java	Encapsula título, descripción, fecha de entrega y enlace al objeto Curso asociado.
Controlador	GestorAcademico.java	Administra las colecciones en memoria, valida unicidad y construye reportes consolidados.
Vista	VentanaPrincipal.java	Renderiza el JFrame Swing, paneles modulares con TitledBorder y listeners de eventos.
Principal	Main.java	Punto de arranque que inicializa el gestor y despliega la UI en el hilo EDT de Swing.

4. Evidencias de Ejecución (Capturas de Pantalla)

A continuación se presentan las evidencias gráficas obtenidas durante las pruebas del sistema en ejecución:

4.1. Maquetación y Registro de Cursos y Tareas

The screenshot displays the main application window with a light blue background. It is divided into two main sections. The left section contains three modules, each with its own form and a button. The right section is a console area for displaying information.

1. MÓDULO DE CURSOS

Código del Curso:

Nombre del Curso:

Tutor / Catedrático:

2. MÓDULO DE TAREAS ACADÉMICAS

Curso Asociado:

Título de la Tarea:

Descripción:

Fecha de Entrega:

3. MÓDULO DE CONVERSIÓN DE TEMPERATURA

Temperatura (°C):

ÁREA DE VISUALIZACIÓN DE INFORMACIÓN

=== BIENVENIDO AL SISTEMA DE GESTIÓN ACADÉMICA ===

Presione 'Mostrar Información Consolidada' para listar los cursos y tareas actualmente registrados en memoria.

Figura 1: Interfaz principal con los paneles de Cursos, Tareas, Conversión y Consola.

4.2. Muestreo de Información Consolidada en JTextArea

1. MÓDULO DE CURSOS

Código del Curso:

0772

Nombre del Curso:

Estructuras de Datos

Tutor / Catedrático:

Ing. Especialista

Registrar Curso

2. MÓDULO DE TAREAS ACADÉMICAS

Curso Asociado:

0770 - Introducción a la Programación 1

Título de la Tarea:

Proyecto 1

Descripción:

Sistema de Refugio de Animales en Java Sw

Fecha de Entrega:

25/09/2026

Agregar Tarea

3. MÓDULO DE CONVERSIÓN DE TEMPERATURA

Temperatura (°C):

Convertir a °F

Convertir a K

Resultado: Ingrese un valor y presione una escala

Mostrar Información Consolidada

Limpiar

=====

REPORTE CONSOLIDADO DE CURSOS Y TAREAS ACADEMICAS

=====

RESUMEN GENERAL:

* Total de Cursos Registrados : 3

* Total de Tareas Asignadas : 3

DETALLE DE CURSOS REGISTRADOS:

[1]Codigo: 0770 | Nombre: Introducción a la Programaci

[2]Codigo: 0960 | Nombre: Matemática para Computación

[3]Codigo: 0771 | Nombre: IPC 1 - Laboratorio |

DETALLE DE TAREAS ACADEMICAS ASIGNADAS:

[1]Tarea: Actividad 5 | Curso: 0770 | Entrega

Descripcion: Construcción de interfaz Swing y modelos F

[2]Tarea: Practica 1 | Curso: 0770 | Entrega

Descripcion: Sistema de control vehicular con arreglos

[3]Tarea: Hoja de Trabajo 1 | Curso: 0960 | Entrega

Descripcion: Tablas de verdad y lógica formal

FIN DEL REPORTE

=====

Figura 2: Despliegue consolidado de cursos y tareas académicas con formato estructurado.

Estudiante: Cristopher Sic — Segundo Semestre 2026

Página 5 de 17

4.3. Conversión Exitosa de Temperatura (°C a °F y K)

Pruebas realizadas con valor base de 25.00 °C aplicando las fórmulas científicas correspondientes:

1. MÓDULO DE CURSOS

Código del Curso:

Nombre del Curso:

Tutor / Catedrático:

Registrar Curso

2. MÓDULO DE TAREAS ACADÉMICAS

Curso Asociado:

Título de la Tarea:

Descripción:

Fecha de Entrega:

Agregar Tarea

3. MÓDULO DE CONVERSIÓN DE TEMPERATURA

Temperatura (°C):

Convertir a °F **Convertir a K**

Resultado: 25.00 °C = 77.00 °F

ÁREA DE VISUALIZACIÓN DE INFORMACIÓN

Mostrar Información Consolidada **Limpiar**

=====

REPORTE CONSOLIDADO DE CURSOS Y TAREAS ACADÉMICAS

=====

RESUMEN GENERAL:

- * Total de Cursos Registrados : 3
- * Total de Tareas Asignadas : 3

DETALLE DE CURSOS REGISTRADOS:

[1]	Codigo: 0770	Nombre: Introducción a la Programaci
[2]	Codigo: 0960	Nombre: Matemática para Computación
[3]	Codigo: 0771	Nombre: IPC 1 - Laboratorio

DETALLE DE TAREAS ACADÉMICAS ASIGNADAS:

[1]	Tarea: Actividad 5	Curso: 0770	Entrega
	Descripción: Construcción de interfaz Swing y modelos F		
[2]	Tarea: Practica 1	Curso: 0770	Entrega
	Descripción: Sistema de control vehicular con arreglos		
[3]	Tarea: Hoja de Trabajo 1	Curso: 0960	Entrega
	Descripción: Tablas de verdad y lógica formal		

=====

FIN DEL REPORTE

=====

Figura 3: Conversión a Fahrenheit: 25.00 °C = 77.00 °F.

1. MÓDULO DE CURSOS

Código del Curso:

Nombre del Curso:

Tutor / Catedrático:

Registrar Curso

2. MÓDULO DE TAREAS ACADÉMICAS

Curso Asociado:

Título de la Tarea:

Descripción:

Fecha de Entrega:

Agregar Tarea

3. MÓDULO DE CONVERSIÓN DE TEMPERATURA

Temperatura (°C):

Convertir a °F **Convertir a K**

Resultado: 25.00 °C = 298.15 K

ÁREA DE VISUALIZACIÓN DE INFORMACIÓN

Mostrar Información Consolidada **Limpiar**

=====

REPORTE CONSOLIDADO DE CURSOS Y TAREAS ACADÉMICAS

=====

RESUMEN GENERAL:

- * Total de Cursos Registrados : 3
- * Total de Tareas Asignadas : 3

DETALLE DE CURSOS REGISTRADOS:

[1]	Codigo: 0770	Nombre: Introducción a la Programaci
[2]	Codigo: 0960	Nombre: Matemática para Computación
[3]	Codigo: 0771	Nombre: IPC 1 - Laboratorio

DETALLE DE TAREAS ACADÉMICAS ASIGNADAS:

[1]	Tarea: Actividad 5	Curso: 0770	Entrega
	Descripción: Construcción de interfaz Swing y modelos F		
[2]	Tarea: Practica 1	Curso: 0770	Entrega
	Descripción: Sistema de control vehicular con arreglos		
[3]	Tarea: Hoja de Trabajo 1	Curso: 0960	Entrega
	Descripción: Tablas de verdad y lógica formal		

=====

FIN DEL REPORTE

=====

Figura 4: Conversión a Kelvin: 25.00 °C = 298.15 K.

4.4. Despliegue de Alertas Modales con JOptionPane

Pruebas de estrés y validación defensiva ante entradas erróneas del usuario:

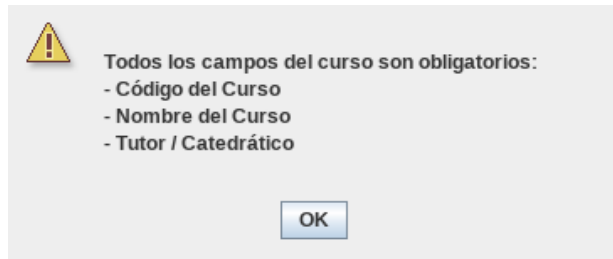


Figura 5: Alerta por omisión de campos obligatorios.

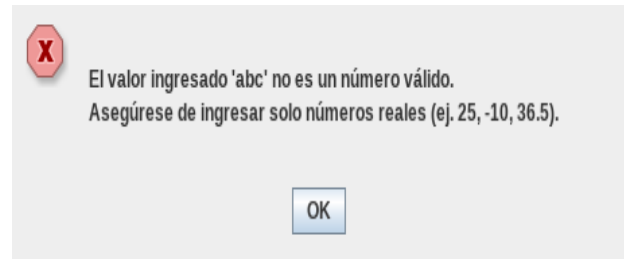


Figura 6: Alerta por formato numérico inválido (try-catch).

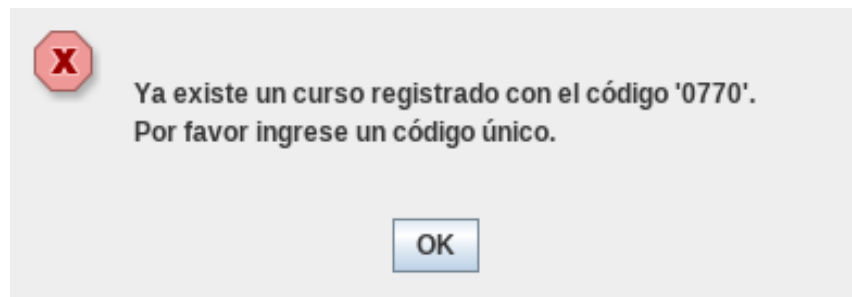


Figura 7: Alerta por intento de registrar un código de curso duplicado.

5. Código Fuente Completo y Comentado

5.1. Clase Curso.java (modelo/Curso.java)

```
package cris.sic.actividad5.modelo;

/**
 * Clase que representa la entidad Curso en el dominio academico.
 * Aplica el principio de encapsulamiento con atributos privados y metodos de acceso.
 */
public class Curso {

    // Identificador unico o codigo del curso (ej. "0770", "IPC1")
    private String codigo;

    // Nombre descriptivo de la materia o asignatura (ej. "Introduccion a la Programacion")
    private String nombre;

    // Nombre completo del catedratico o tutor a cargo del curso
    private String tutor;

    /**
     * Constructor por defecto sin parametros.
     */
    public Curso() {
    }

    /**
     * Constructor con todos los parametros para inicializar el objeto Curso.
     *
     * @param codigo Identificador unico del curso
     * @param nombre Nombre descriptivo del curso
     * @param tutor Nombre del docente o tutor asignado
     */
    public Curso(String codigo, String nombre, String tutor) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.tutor = tutor;
    }

    // =====
    // METODOS GETTERS Y SETTERS (ENCAPSULAMIENTO)
    // =====

    /**
     * Obtiene el codigo del curso.
     * @return Codigo unico en formato String
     */
    public String getCodigo() {
        return codigo;
    }

    /**
     * Asigna o actualiza el codigo del curso.
     * @param codigo Nuevo codigo del curso
     */
    public void setCodigo(String codigo) {
        this.codigo = codigo;
    }

    /**
     * Obtiene el nombre del curso.
     * @return Nombre del curso
     */
    public String getNombre() {
        return nombre;
    }
}
```

```

    }

    /**
     * Asigna o actualiza el nombre del curso.
     * @param nombre Nuevo nombre del curso
     */
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    /**
     * Obtiene el tutor asignado al curso.
     * @return Nombre del tutor o catedratico
     */
    public String getTutor() {
        return tutor;
    }

    /**
     * Asigna o actualiza el tutor asignado al curso.
     * @param tutor Nuevo tutor a cargo
     */
    public void setTutor(String tutor) {
        this.tutor = tutor;
    }

    // =====
    // METODOS DESCRIPTIVOS Y DE FORMATO
    // =====

    /**
     * Retorna una ficha estilizada con los datos completos del curso.
     * Utilizado para reportes y visualizacion en consola o interfaces graficas.
     *
     * @return Cadena formateada con la informacion del curso
     */
    public String getInformacion() {
        return "=====\n"
            + "Codigo : " + (codigo != null ? codigo : "N/A") + "\n"
            + "Curso  : " + (nombre != null ? nombre : "N/A") + "\n"
            + "Tutor   : " + (tutor != null ? tutor : "N/A") + "\n"
            + "=====";
    }

    /**
     * Representacion amigable en cadena de texto del objeto Curso.
     * Ideal para ser renderizado dentro de componentes Swing como JComboBox o JList.
     *
     * @return Cadena descriptiva: "CODIGO - NOMBRE"
     */
    @Override
    public String toString() {
        return codigo + " - " + nombre;
    }
}

```

5.2. Clase TareaAcademica.java (modelo/TareaAcademica.java)

```
package cris.sic.actividad5.modelo;

/**
 * Clase que modela una Tarea Academica asignada a un curso especifico.
 * Aplica el principio de encapsulamiento mediante modificadores de acceso privados y metodos getter/setter.
 */
public class TareaAcademica {

    // Referencia al objeto Curso al cual pertenece la tarea academica
    private Curso cursoAsociado;

    // Titulo descriptivo de la actividad o entrega (ej. "Practica 1", "Investigacion P00")
    private String titulo;

    // Descripcion detallada o instrucciones de la tarea
    private String descripcion;

    // Fecha limite de entrega en formato texto (ej. "15/09/2026")
    private String fechaEntrega;

    /**
     * Constructor por defecto.
     */
    public TareaAcademica() {
    }

    /**
     * Constructor completo para instanciar una tarea con todos sus atributos requeridos.
     *
     * @param cursoAsociado Objeto Curso al que pertenece la tarea
     * @param titulo Titulo o nombre de la tarea
     * @param descripcion Detalle o enunciado de la asignacion
     * @param fechaEntrega Fecha limite para completar la tarea
     */
    public TareaAcademica(Curso cursoAsociado, String titulo, String descripcion, String fechaEntrega) {
        this.cursoAsociado = cursoAsociado;
        this.titulo = titulo;
        this.descripcion = descripcion;
        this.fechaEntrega = fechaEntrega;
    }

    // =====
    // METODOS GETTERS Y SETTERS (ENCAPSULAMIENTO)
    // =====

    /**
     * Obtiene el curso asociado a esta tarea.
     * @return Objeto de tipo Curso
     */
    public Curso getCursoAsociado() {
        return cursoAsociado;
    }

    /**
     * Asigna o reasocia la tarea a un nuevo curso.
     * @param cursoAsociado Nuevo objeto Curso
     */
    public void setCursoAsociado(Curso cursoAsociado) {
        this.cursoAsociado = cursoAsociado;
    }

    /**
     * Obtiene el titulo de la tarea academica.
     * @return Titulo en formato String
     */
}
```

```

public String getTitulo() {
    return titulo;
}

/**
 * Asigna el titulo de la tarea academica.
 * @param titulo Nuevo titulo de la tarea
 */
public void setTitulo(String titulo) {
    this.titulo = titulo;
}

/**
 * Obtiene la descripcion o instrucciones de la tarea.
 * @return Descripcion en formato String
 */
public String getDescripcion() {
    return descripcion;
}

/**
 * Asigna la descripcion de la tarea.
 * @param descripcion Detalle o texto explicativo
 */
public void setDescripcion(String descripcion) {
    this.descripcion = descripcion;
}

/**
 * Obtiene la fecha limite de entrega.
 * @return Fecha en formato String
 */
public String getFechaEntrega() {
    return fechaEntrega;
}

/**
 * Asigna la fecha limite de entrega.
 * @param fechaEntrega Fecha de entrega
 */
public void setFechaEntrega(String fechaEntrega) {
    this.fechaEntrega = fechaEntrega;
}

// =====
// METODOS DESCRIPTIVOS Y DE FORMATO
// =====

/**
 * Genera un reporte detallado con toda la informacion de la tarea,
 * incluyendo el curso al que fue asignada.
 *
 * @return Cadena con formato estructurado
 */
public String getDetalles() {
    String infoCurso = (cursoAsociado != null)
        ? cursoAsociado.getCodigo() + " - " + cursoAsociado.getNombre()
        : "Sin curso asignado";

    return "-----\n"
        + " Tarea      : " + (titulo != null ? titulo : "Sin titulo") + "\n"
        + " Curso       : " + infoCurso + "\n"
        + " Entrega     : " + (fechaEntrega != null ? fechaEntrega : "No especificada") + "\n"
        + " Descripcion : " + (descripcion != null ? descripcion : "Sin descripcion") + "\n"
        + "-----";
}
/**

```

```
    * Representacion textual concisa de la tarea.
    *
    * @return Cadena con el titulo y la fecha de entrega
    */
    @Override
    public String toString() {
        return titulo + " (Entrega: " + fechaEntrega + ")";
    }
}
```

5.3. Clase GestorAcademico.java (controlador/GestorAcademico.java)

```
package cris.sic.actividad5.controlador;

import cris.sic.actividad5.modeloCurso;
import cris.sic.actividad5.modelo.TareaAcademica;

import java.util.ArrayList;

/**
 * Controlador de negocio y gestor de colecciones para Cursos y Tareas Academicas.
 * Centraliza las operaciones CRUD en memoria mediante arreglos dinamicos (ArrayList).
 */
public class GestorAcademico {

    // Coleccion dinamica en memoria para almacenar los cursos registrados
    private ArrayList<Curso> listaCursos;

    // Coleccion dinamica en memoria para almacenar las tareas academicas creadas
    private ArrayList<TareaAcademica> listaTareas;

    /**
     * Constructor que inicializa las listas en memoria vacias.
     */
    public GestorAcademico() {
        this.listaCursos = new ArrayList<>();
        this.listaTareas = new ArrayList<>();
    }

    // =====
    // OPERACIONES SOBRE CURSOS
    // =====

    /**
     * Registra un nuevo curso en la coleccion verificando que el codigo no se encuentre duplicado.
     *
     * @param curso Objeto Curso a registrar
     * @return true si se registro satisfactoriamente, false si ya existe un curso con el mismo codigo
     */
    public boolean registrarCurso(Curso curso) {
        if (curso == null || curso.getCodigo() == null || curso.getCodigo().trim().isEmpty()) {
            return false;
        }

        // Validar unicidad de codigo
        if (buscarCursoPorCodigo(curso.getCodigo().trim()) != null) {
            return false; // Codigo duplicado
        }

        return listaCursos.add(curso);
    }

    /**
     * Busca y retorna un Curso a partir de su codigo identificador (comparacion sin distincion de mayusculas).
     *
     * @param codigo Identificador del curso a localizar
     * @return El objeto Curso coincidente o null si no se encuentra
     */
    public Curso buscarCursoPorCodigo(String codigo) {
        if (codigo == null) return null;
        for (Curso c : listaCursos) {
            if (c.getCodigo().equalsIgnoreCase(codigo.trim())) {
                return c;
            }
        }
        return null;
    }
}
```

```
/**
 * Retorna la lista completa de cursos registrados en el sistema.
 * @return ArrayList con los objetos Curso
 */
public ArrayList<Curso> getCursos() {
    return listaCursos;
}

/**
 * Retorna la cantidad total de cursos registrados en la lista.
 * @return Numero entero de cursos
 */
public int getTotalCursos() {
    return listaCursos.size();
}

// =====
// OPERACIONES SOBRE TAREAS ACADEMICAS
// =====

/**
 * Registra una nueva tarea academica vinculada a un curso.
 *
 * @param tarea Objeto TareaAcademica a incorporar en la coleccion
 * @return true si la insercion fue exitosa, false en caso contrario
 */
public boolean registrarTarea(TareaAcademica tarea) {
    if (tarea == null || tarea.getCursoAsociado() == null) {
        return false;
    }
    return listaTareas.add(tarea);
}

/**
 * Retorna la lista completa de tareas academicas registradas.
 * @return ArrayList con los objetos TareaAcademica
 */
public ArrayList<TareaAcademica> getTareas() {
    return listaTareas;
}

/**
 * Filtra y retorna las tareas academicas que corresponden a un curso especifico.
 *
 * @param codigoCurso Codigo del curso objetivo
 * @return ArrayList con las tareas pertenecientes al curso
 */
public ArrayList<TareaAcademica> obtenerTareasPorCurso(String codigoCurso) {
    ArrayList<TareaAcademica> filtradas = new ArrayList<>();
    if (codigoCurso == null) return filtradas;

    for (TareaAcademica t : listaTareas) {
        if (t.getCursoAsociado() != null && t.getCursoAsociado().getCodigo().equalsIgnoreCase(codigoCurso.trim())) {
            filtradas.add(t);
        }
    }
    return filtradas;
}

/**
 * Retorna la cantidad total de tareas academicas registradas.
 * @return Numero entero de tareas
 */
public int getTotalTareas() {
    return listaTareas.size();
}

// =====
```

```

// METODOS DE GENERACION DE REPORTE
// =====

/**
 * Recorre las colecciones de cursos y tareas para construir un reporte
 * consolidado en formato de texto legible, ideal para presentarse en el JTextArea.
 *
 * @return Cadena con el reporte estructurado completo
 */
public String generarReporteConsolidado() {
    StringBuilder sb = new StringBuilder();
    sb.append("=====\n");
    sb.append("          REPORTE CONSOLIDADO DE CURSOS Y TAREAS ACADEMICAS          \n");
    sb.append("=====\n\n");

    sb.append("RESUMEN GENERAL:\n");
    sb.append(" * Total de Cursos Registrados : ").append(listaCursos.size()).append("\n");
    sb.append(" * Total de Tareas Asignadas   : ").append(listaTareas.size()).append("\n\n");

    sb.append("-----\n");
    sb.append("DETALLE DE CURSOS REGISTRADOS:\n");
    sb.append("-----\n");

    if (listaCursos.isEmpty()) {
        sb.append(" (No hay cursos registrados en el sistema actualmente)\n\n");
    } else {
        for (int i = 0; i < listaCursos.size(); i++) {
            Curso c = listaCursos.get(i);
            sb.append(String.format(" [%d] Código: %-8s | Nombre: %-25s | Tutor: %s\n",
                (i + 1), c.getCodigo(), c.getNombre(), c.getTutor()));
        }
        sb.append("\n");
    }

    sb.append("-----\n");
    sb.append("DETALLE DE TAREAS ACADEMICAS ASIGNADAS:\n");
    sb.append("-----\n");

    if (listaTareas.isEmpty()) {
        sb.append(" (No hay tareas registradas en el sistema actualmente)\n\n");
    } else {
        for (int i = 0; i < listaTareas.size(); i++) {
            TareaAcademica t = listaTareas.get(i);
            String codCurso = (t.getCursoAsociado() != null) ? t.getCursoAsociado().getCodigo() : "N/A";
            sb.append(String.format(" [%d] Tarea: %-20s | Curso: %-8s | Entrega: %s\n",
                (i + 1), t.getTitulo(), codCurso, t.getFechaEntrega()));
            sb.append("      Descripción: ").append(t.getDescripcion()).append("\n");
        }
        sb.append("\n");
    }

    sb.append("=====\n");
    sb.append("                      FIN DEL REPORTE                      \n");
    sb.append("=====\n");

    return sb.toString();
}
}

```

5.4. Clase Main.java (Main.java)

```
package cris.sic.actividad5;

import cris.sic.actividad5.controlador.GestorAcademico;
import cris.sic.actividad5.modeloCurso;
import cris.sic.actividad5.vista.VentanaPrincipal;

import javax.swing.SwingUtilities;

/**
 * Punto de entrada principal de la aplicacion para la Actividad 5.
 * Inicializa la aplicacion e invoca la interfaz grafica VentanaPrincipal en el hilo EDT de Swing.
 */
public class Main {

    /**
     * Metodo principal de ejecucion de la aplicacion.
     *
     * @param args Argumentos de linea de comandos
     */
    public static void main(String[] args) {
        // Inicializar el controlador en memoria
        GestorAcademico gestor = new GestorAcademico();

        // Cursos iniciales de cortesia para facilitar la interaccion inmediata en la interfaz
        Curso c1 = new Curso("0770", "Introducción a la Programación 1", "Ing. Catedrático");
        Curso c2 = new Curso("0960", "Matemática para Computación 1", "Lic. Catedrático");
        gestor.registrarCurso(c1);
        gestor.registrarCurso(c2);

        // Lanzamiento seguro de la interfaz grafica en el Event Dispatch Thread (EDT) de Swing
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                VentanaPrincipal ventana = new VentanaPrincipal(gestor);
                ventana.setVisible(true);
            }
        });
    }
}
```

6. Conclusiones

- 1. Consolidación de Principios POO:** La correcta aplicación de abstracción y encapsulamiento protegió la integridad de las entidades Curso y TareaAcademica, permitiendo desacoplar la representación de los datos de su visualización.
- 2. Dominio de Java Swing sin Diseñadores Visuales:** La construcción de la interfaz por código puro demostró que es posible lograr distribuciones elegantes, perfectamente alineadas y mantenibles sin generar dependencias de código propietario o drag-and-drop.
- 3. Robustez mediante Programación Defensiva:** La combinación de validaciones preventivas de cadenas con la captura de excepciones (NumberFormatException) garantiza que la aplicación no colapse ante entradas erróneas del usuario, ofreciendo una experiencia interactiva confiable.
- 4. Escalabilidad con Colecciones Dinámicas:** El empleo de ArrayList en el GestorAcademico permitió gestionar volúmenes variables de información en memoria de manera flexible, facilitando la adición y filtrado de registros en tiempo de ejecución.

7. Referencias Bibliográficas

1. Oracle Corporation. (2024). *Java Platform, Standard Edition Documentation (JDK 21)*. Oracle Technology Network.
2. Deitel, P., & Deitel, H. (2018). *Java: How to Program (Early Objects)* (11.^a ed.). Pearson Education.
3. Eckel, B. (2006). *Thinking in Java* (4.^a ed.). Prentice Hall.
4. Horstmann, C. S., & Cornell, G. (2019). *Core Java Volume I — Fundamentals* (11.^a ed.). Prentice Hall.